

# D2100R0

## Keep `unhandled_exception` of a promise type mandatory - a response to US062 and FR066

Published Proposal, 2020-02-10

**This version:**

<http://wg21.link/P2100>

**Author:**

[Michał Dominiak](#) (NVIDIA)

**Audience:**

EWG

**Project:**

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

---

## Table of Contents

- 1 Abstract
- 2 US062
- 3 FR066
- 4 How do I tell if I need the function to be defined?

### § 1. Abstract

This paper is a response to two of the NB comments filed against the C++20 CD, and is a call to reject them both and apply no changes to the working draft as a response to US062 and FR066. Both of these comments attempt to change the fact that `unhandled_exception` is always a required member of a

coroutine promise type, and this paper argues that we should instead ship the status quo and make possible further adjustments in the future revisions of the standard (in a backwards-compatible manner).

[US062](#) is an NB comment that requests that `noexcept` on a coroutine declaration be treated as a request to not have any exception handling inside the coroutine body, and therefore make it not require the promise type to define an `unhandled_exception` member function. This was motivated by the desire to be able to ship an implementation of C++20 coroutines for an environment that has no exceptions support, **but also** for an environment that supports exceptions only partially. (The author of this paper feels it is justified for him to state this as a fact, because he was present while the comment was being drafted.)

[FR066](#) proposes to "bless" rethrowing the exception from the `unhandled_exception` member function of a coroutine promise type as the default exception handling mechanism for coroutines. The motivation appears to be to reduce the amount of code that an author of a promise type needs to write when writing such a class.

This paper will attempt to explain why:

1. the proposed resolution to US062 was misguided;
2. the status quo of C++20 is acceptable to the authors of US062; and
3. the proposed resolution of FR066 would make the text less acceptable to the authors of US062 than the status quo.

## § 2. US062

The US comment on this topic assumed a larger meaning of `noexcept` when applied to a coroutine definition than currently exists. Right now (modulo a wording bug that the author of this paper believes is currently being fixed by CWG), `noexcept` applies only to the bootstrapping code of a coroutine - i.e. if there's an exception throw into the coroutine body while the coroutine frame is allocated, or while the `initial_suspend` function is called, `std::terminate` is invoked. That's it. After the coroutine is initially suspended, `noexcept` no longer applies.

This is consistent with the view of the world that whether a function is a coroutine or not is just an implementation detail. Currently (assuming a library that provides futures - or a similar construct - with a way to attach a continuation (spelled `.then` in this example), in the following code:

```
future<T> do_a_thing() noexcept
{
    a();
    return b().then(c);
};
```

if `a` or `b` throw an exception, `std::terminate` is called; but if `c` calls an exception, that is not the case, and the exception is propagated as needed. Therefore, if there should be a mechanism for specifying that the compiler-generated portion of a coroutine's body should not handle any exceptions, it should **not** be the `noexcept` specifier on the declaration of the coroutine itself; it is imaginable that some other mechanism could be used to specify this, and the author of this paper went on a short journey of trying to specify such an additional customization point for coroutines. It is possible to do that, but the author no longer believes that that is the best way to solve the original concern behind this NB comment. It would also be a pure extension of the current specification.

To attempt to quickly explain the development environment of CUDA: your source code is compiled twice\*: once for the "host" (CPU) target, and once for the "device" (GPU) target. There's several differences between those two targets, but the most significant one is that while the host target can support C++ exceptions, the device target does not. This is a bit problematic given the current status quo of C++, because the natural definition of the `unhandled_exception` function is ill-formed on the device target.

(\* this is not entirely accurate; it's really "more than twice", if you specify more than a single GPU architecture target, but the author wanted to focus on the simplest case here. This footnote is provided for completeness only.)

However, the current practice of multiple (all?) coroutine implementations is that in the - non-standard-compliant - mode of `-fno-exceptions`, the requirement of the promise type having an `unhandled_exception` member is waived, and no exception handling code is generated where the standard requires it. While discussing possible ways to address this comment with other parties, we've come to the conclusion that this is an acceptable implementation strategy for us:

1. While generating code for the device side, our compiler can employ the strategy of not enforcing the requirement of the `unhandled_exception` member of the promise type being provided.
2. When writing a coroutine promise type that is meant to be aware of exceptions, a programmer can make sure that their `unhandled_exception` function is marked as host-only.

### § 3. FR066

The FR comment on this issue requests to make `unhandled_exception` default to a definition equivalent to this one:

```
void unhandled_exception()  
{  
    throw;  
}
```

This has three consequences, one directly intended and rather positive, and two the author of this paper is ambivalent about:

1. Authors of coroutine promise types that don't want to be aware of exceptions existing can ignore their existence. (Intended.)
2. Rethrowing the exception (as opposed to either (a) terminating the program or (b) passing it to someplace else) becomes the "blessed" option for handling exceptions. This means that we consider it to be the option that'd be employed by the **majority** of coroutine promise types (and the author of this paper is not convinced this is accurate).
3. Subtle bugs can be hidden. Given the current status quo (and the current implementations of C++20 coroutines), if a programmer implements a coroutine promise type in an environment with exceptions disabled (and therefore provides no `unhandled_exception` member function), and someone attempts to use such a type in an environment with exceptions enabled, they get a compiler error. If the proposed resolution to this comment is applied, their code will compile, but perhaps with unintended runtime behavior in the presence of uncaught exceptions.

The third consequence here is why the author of this paper considers the proposed resolution of FR066 to be a step in the wrong direction, particularly for the Nvidia CUDA platform, where the anticipated amount of code that needs to work in an environment that mixes targets where exceptions are enabled with ones where they are disabled appears to be larger than in other environments.

### § 4. How do I tell if I need the function to be defined?

If the status quo is kept, it may become necessary to test for whether exceptions are enabled or not. From the point of view of the author of the paper, it is fine. Currently, exceptions are a required language feature, so from the point of view of the standard there's nothing to test for. However, for completeness, the rest of this section explains what tools a programmer can use to make sure their `unhandled_exception` is only defined for targets supporting exceptions.

When writing CUDA code, the following will compile (assuming the host compiler has exceptions enabled):

```
__host__ // mark the function as targeting only the CPU
void unhandled_exception()
{
    throw;
}
```

GCC provides a predefined macro called `__cpp_exceptions` when exceptions are enabled, and doesn't when they are disabled.

Clang already provides a way to query for exception support (`__has_feature(cxx_exceptions)`).

Under other implementations, the programmer can use the build system to define a macro in addition to setting the `-fno-exceptions` flag.

Finally, if the greater "make freestanding more useful" effort succeeds in making exceptions optional, the author expects it to also provide a standardized way of checking for exception support being enabled or not.